Skill 1_2520090235_ossp

Session1:

Task 1:

```
  GNU nano 8.7.1
#include <stdio.h>

int main() {
    printf("Hello, OSSP Skill Week 1!\n");
    printf("GCC compilation working successfully.\n");

    return 0;
}
```

Output:

```
rohita@LaxmiDurga:~$ nano s1.c
rohita@LaxmiDurga:~$ gcc s1.c
rohita@LaxmiDurga:~$ ./a.out
Hello, OSSP Skill Week 1!
```

Task 2: To Analyse process abstraction, execute fork(), understand exec() family, analyse parent-child relationships, inspect process tree, practice system call tracing.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Executing ls using execlp()\n");

        execlp("ls", "ls", "-l", NULL);

        perror("exec failed");
        exit(1);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

Output

```
Parent Process Started
Parent PID: 1354

Parent Process
Parent PID: 1354

Child Process
Child PID: 1355
Parent waiting for child...
Child PID: 1355
Parent PID: 1354
Executing ls using execlp()
total 76
-rw-r--r--  1 rohita rohita    57 Aug  6 14:47 First.c
drwxr-xr-x  3 rohita rohita  4096 Aug  3 05:02 Hello-World
drwxr-xr-x 11 rohita rohita  4096 Jul 30 09:39 OSSP
-rw-r--r--  1 rohita rohita  1206 Aug  7 15:18 Practical2.c
-rw-r--r--  1 rohita rohita   935 Aug  6 15:00 Skill2.c
-rw-r--r--  1 rohita rohita   654 Aug  7 14:32 Skill3.c
-rw-r--r--  1 rohita rohita   802 Aug  7 14:48 Skill3Task2.c
-rwxr-xr-x  1 rohita rohita 16288 Aug 19 23:47 a.out
-rw-r--r--  1 rohita rohita   936 Aug  5 16:22 command_exec.c
-rw-------  1 rohita rohita     1 Jul 28 14:48 nano.565.save
-rw-r--r--  1 rohita rohita   156 Aug 18 07:00 os1
drwxr-xr-x  6 rohita rohita  4096 Aug  4 08:12 ossp
-rw-r--r--  1 rohita rohita   148 Aug 19 09:50 s1.c
-rw-r--r--  1 rohita rohita   881 Aug 19 23:47 s12.c
-rw-r--r--  1 rohita rohita  1905 Aug 11 07:08 skil4Task1.c
-rw-r--r--  1 rohita rohita   968 Aug  7 14:52 skill3Task2.c
Child process completed.
```

Session2:

Task1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

Code:

```
 GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main() {
    char command[100];

    printf("OSSP Interactive Shell\n");

    while (1) {
        printf("myshell> ");

        if (fgets(command, sizeof(command), stdin) == NULL) {
            break;
        }

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(command) == 0) {
            continue;
        }

        printf("You entered: %s\n", command);
    }

    return 0;
}
```

Output:

```
OSSP Interactive Shell
myshell> ls
You entered: ls
myshell> pwd
You entered: pwd
myshell>
```

Task2 : To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```c
#include <stdio.h>

int main() {
    char buffer[100];
    int ch;
    int index = 0;

    printf("Type a command and press Enter:\n");
    printf("input > ");

    while ((ch = getchar()) != '\n' && ch != EOF) {
        if (ch == 127 || ch == 8) {
            if (index > 0) {
                index--;
            }
        } else if (index < 99) {
            buffer[index++] = ch;
        }
    }

    buffer[index] = '\0';

    printf("\nCommand entered: %s\n", buffer);
    printf("Characters stored: %d\n", index);

    return 0;
}
```

Output:

```
Type a command and press Enter:
input > ls

Command entered: ls
Characters stored: 2
```